

6th Semester AI Introduction to Deep Learning

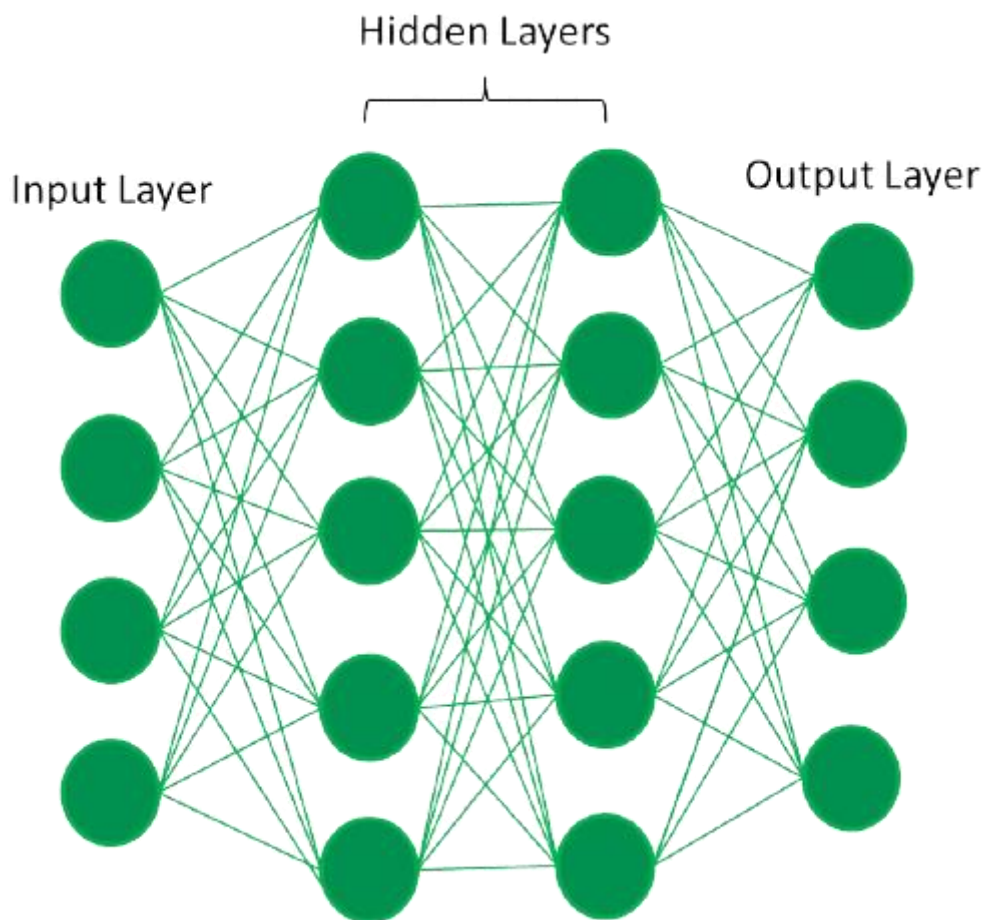
Deep Learning is transforming the way machines understand, learn and interact with complex data. Deep learning mimics neural networks of the human brain, it enables computers to autonomously uncover patterns and make informed decisions from vast amounts of unstructured data.

How Deep Learning Works?

Neural network consists of layers of interconnected nodes or neurons that collaborate to process input data. In a fully connected deep neural network data flows through multiple layers where each neuron performs nonlinear transformations, allowing the model to learn intricate representations of the data.

In a deep neural network the input layer receives data which passes through hidden layers that transform the data using nonlinear functions. The final output layer generates the model's prediction.

For more details on neural networks refer to: [What is a Neural Network?](#)



Fully Connected Deep Neural Network

Difference between Machine Learning and Deep Learning

Machine learning and Deep Learning both are subsets of artificial intelligence but there are many similarities and differences between them.

Machine Learning	Deep Learning
Apply statistical algorithms to learn the hidden patterns and relationships in the dataset.	Uses artificial neural network architecture to learn the hidden patterns and relationships in the dataset.
Can work on the smaller amount of dataset	Requires the larger volume of dataset compared to machine learning
Better for the low-label task.	Better for complex task like image processing, natural language processing, etc.
Takes less time to train the model.	Takes more time to train the model.
A model is created by relevant features which are manually extracted from images to detect an object in the image.	Relevant features are automatically extracted from images. It is an end-to-end learning process.
Less complex and easy to interpret the result.	More complex, it works like the black box interpretations of the result are not easy.
It can work on the CPU or requires less computing power as compared to deep learning.	It requires a high-performance computer with GPU.

Types of Deep Learning – Detailed Explanation

Deep Learning methods can be categorized based on **how models learn from data**. The major types are:

1. **Supervised Deep Learning**
2. **Unsupervised Deep Learning**
3. **Semi-Supervised Deep Learning**
4. **Self-Supervised Deep Learning**
5. **Reinforcement Learning (Deep RL)**
6. **Transfer Learning.**

Types of Deep Learning

● 1. Supervised Deep Learning

In supervised learning, the system learns using a **labeled dataset**—each input has a corresponding target output.

The model's goal is to **learn a mapping** from input → output.

✓ Key Characteristics

- Requires large labeled datasets
- High accuracy for prediction tasks
- Loss function measures error between predicted and actual labels
- Backpropagation is used for learning

✓ Applications

- Image classification (face recognition, defect detection)
- Speech-to-text systems
- Medical diagnosis
- Spam email detection

✓ Common Deep Learning Models

- **Convolutional Neural Networks (CNNs)** – for images
 - **Recurrent Neural Networks (RNNs)/LSTMs** – for sequences
 - **Deep Feedforward Networks** – for structured data
-

● 2. Unsupervised Deep Learning

Unsupervised learning works with **unlabeled data**, meaning only inputs are given.

The model identifies **hidden patterns, structures, or representations** without guidance.

✓ Key Characteristics

- No ground truth labels
- Useful for exploring unknown patterns
- Learns data distribution

✓ Applications

- Clustering similar images/documents
- Anomaly/fraud detection
- Data compression
- Noise removal

✓ Common Models

- **Autoencoders** – compress and reconstruct data
 - **Generative Adversarial Networks (GANs)** – generate new data samples
 - **Restricted Boltzmann Machines (RBMs)** – probabilistic models
-

● 3. Semi-Supervised Deep Learning

This method uses a **small amount of labeled data plus a large amount of unlabeled data**.

It combines advantages of both supervised and unsupervised learning.

✓ Why Needed?

Labeling data is expensive (e.g., medical images).

Semi-supervised makes learning possible with minimal labeled samples.

✓ Key Characteristics

- Model learns structure using unlabeled data
- Fine-tunes using labeled data
- Improves accuracy over strict unsupervised learning

✓ Applications

- Text classification
- Sentiment analysis
- Medical image processing
- Speech recognition

✓ Common Models

- Semi-supervised **GANs**
 - Semi-supervised **autoencoders**
 - Graph-based neural networks
-

● 4. Self-Supervised Deep Learning

Self-supervised learning is a subset of unsupervised learning.

Here, the model **creates its own labels** from the input data.

✓ How it works?

A “pretext task” is created—for example:

- Predict missing words in a sentence
- Predict the next video frame
- Predict masked parts of an image

The model learns powerful internal representations.

✓ Key Characteristics

- No manual labeling required
- Very effective for NLP and vision
- Forms foundation of large language models

✓ Applications

- NLP (BERT, GPT models)
- Vision Transformers (ViT)
- Speech processing

✓ Common Models

- **Transformers** (BERT, GPT, RoBERTa)
- **Masked Autoencoders (MAE)**
- Contrastive Learning Models (SimCLR, MoCo)

● 5. Reinforcement Learning (Deep Reinforcement Learning)

In Reinforcement Learning, an **agent** learns to take actions in an **environment** to maximize **reward**.

When neural networks are used for function approximation, it becomes **Deep Reinforcement Learning (DRL)**.

✓ Key Characteristics

- Learning by trial-and-error
- No labeled data; only rewards/penalties
- Suitable for sequential decision-making
- Exploration vs exploitation dilemma

✓ Applications

- Robotics (navigation, manipulation)
- Autonomous vehicles
- Game playing (Chess, Go, Atari, Dota 2)
- Resource optimization

✓ Common Models

- Deep Q-Networks (DQN)
 - Policy Gradient Methods
 - Actor-Critic Models
 - DDPG, PPO, A3C
-

● 6. Transfer Learning

Transfer learning reuses a **pretrained deep learning model** on a new but related task.

✓ Why used?

Training large models from scratch is costly and needs huge data.
Transfer learning allows:

- Faster training
- Higher accuracy with small datasets

✓ How it works?

1. Take a model pretrained on a large dataset
2. Replace output layers
3. Fine-tune on new dataset

✓ Applications

- Medical imaging
- Object detection
- NLP tasks like sentiment analysis

✓ Common Models

- Pretrained CNNs: **VGG, ResNet, MobileNet**
- Pretrained Transformers: **BERT, GPT,**

History of Deep Learning – Detailed Notes

Deep Learning is a subset of Artificial Intelligence (AI) and Machine Learning (ML) that uses **multi-layered neural networks** to learn complex patterns. Its history spans more than 70 years, evolving through periods of breakthroughs, challenges, and rapid growth.

📌 1. Early Foundations (1940s–1960s)

This period laid the groundwork for neural networks.

1943 – McCulloch & Pitts Model

- Warren McCulloch and Walter Pitts proposed the **first mathematical model of a neuron**.
- Introduced the idea that networks of neurons can compute logical functions.
- Marked the birth of **artificial neural networks (ANNs)**.

1949 – Hebbian Learning

- Donald Hebb proposed the rule: *“Neurons that fire together, wire together.”*
- First theory for how neural connections strengthen—basis for **learning in neural networks**.

1958 – Rosenblatt’s Perceptron

- Frank Rosenblatt developed the **Perceptron**, the first trainable neural network.
- Capable of learning simple patterns (linear separable data).
- Inspired early optimism in machine learning.

1960 – Adaline & Madaline

- Developed by Bernard Widrow and Marcian Hoff.
 - Used **Adaptive Linear Neuron** for pattern recognition.
 - First practically deployed neural network (used in telephone line noise cancellation).
-

📌 2. AI Winter and Limitations (1969–1980s)

During this period, neural networks faced criticism and funding cuts.

1969 – Minsky & Papert’s Criticism

- In their book *Perceptrons*, Marvin Minsky and Seymour Papert showed:
 - Perceptrons cannot learn XOR problem
 - Single-layer networks have severe limitations
- Led to reduced research funding for neural networks → **First AI Winter**.

1970s – Slow Progress

- Limited computational power
- No effective training algorithm for multi-layer networks
- Interest shifted toward symbolic AI and expert systems.

❏ 3. Breakthrough: Backpropagation (1980s)

The revival of neural networks begins.

1986 – Backpropagation Popularized

- David Rumelhart, Geoffrey Hinton, and Ronald Williams published the **backpropagation algorithm**.
- Allowed training of **multi-layer neural networks** effectively.
- Enabled deeper networks and renewed interest.

Hopfield Networks (1982)

- John Hopfield developed **recurrent associative memory networks**.
- Important for optimization & energy-based models.

Boltzmann Machines (1985)

- Developed by Hinton and Sejnowski.
- Introduced probabilistic generative neural models.

❏ 4. The 1990s – The Pre-Deep Learning Era

Research progressed slowly but steadily.

1995 – Support Vector Machines (SVMs)

- Became dominant in ML, outperforming early neural networks.

CNN & LSTM Emergence

- **1989–1998 – Convolutional Neural Networks (CNNs)**
 - Yann LeCun developed LeNet-5 (1998) for digit recognition.
 - Used convolution + backpropagation.
 - Deployed in postal mail sorting.
- **1997 – LSTM (Long Short-Term Memory)**
 - Developed by Hochreiter & Schmidhuber.
 - Solved vanishing gradient problem for sequence learning.
 - Later crucial for speech, language, and video tasks.

Why Deep Learning didn't grow yet?

- Not enough data
- Lack of GPU computation
- Neural networks still hard to train

5. Deep Learning Revolution (2006–2012)

2006 – Hinton’s Deep Belief Networks

- Geoffrey Hinton introduced **Deep Belief Networks (DBNs)**.
- Marked official use of the term “Deep Learning”.
- Showed training deeper networks was possible.

2009 – GPU Acceleration

- Researchers used GPUs for neural network training.
- Boosted training speed by 10–100×.

2012 – ImageNet Breakthrough (AlexNet)

- Created by Alex Krizhevsky, Ilya Sutskever, and Hinton.
- Deep CNN (8 layers) trained on ImageNet.
- Reduced top-5 error rate dramatically.
- Signaled the beginning of modern Deep Learning boom.

6. Modern Deep Learning Era (2013–present)

2014 – Generative Adversarial Networks (GANs)

- Ian Goodfellow introduced GANs.
- Able to generate realistic images, audio, and video.
- Major breakthrough in generative AI.

2015 – ResNet

- He et al. introduced **Residual Networks (ResNet)** with skip connections.
- Allowed very deep networks (152+ layers).
- Solved vanishing gradient issues.

2017 – Transformers

- Vaswani et al. introduced **Attention Is All You Need**.
- Eliminated RNN; enabled parallel processing.
- Basis of modern NLP systems.

2018–2023 – Large Language Models (LLMs)

- BERT (2018)
- GPT series (2018–2024)
- T5, RoBERTa, XLNet

- Reinforced with self-supervised learning + massive data.

2024–Present – Multimodal AI

- Models that understand text, images, audio, video, 3D.
- Foundation models with billions of parameters.
- Deep Learning enters daily applications:
 - Chatbots
 - Medical diagnosis
 - Autonomous driving
 - Generative art and video

7. Key Factors Enabling Deep Learning Boom

1. **Massive datasets** (ImageNet, Common Crawl)
2. **High-speed GPUs & TPUs**
3. **Improved algorithms** (ReLU, BatchNorm, Dropout)
4. **Open-source frameworks** (TensorFlow, PyTorch)

Model of Artificial Neuron – Detailed Notes

The **Artificial Neuron** (also called **Artificial Neural Unit** or **Perceptron unit**) is the fundamental building block of Artificial Neural Networks (ANNs). It is inspired by the structure and functioning of biological neurons in the human brain.

1. Introduction

An **artificial neuron** is a mathematical function that:

- receives multiple inputs
- processes them
- produces an output based on an activation function

It simulates how a biological neuron integrates signals from various other neurons and decides whether to fire or not.

2. Biological Inspiration

A biological neuron consists of:

- **Dendrites** → receive signals
- **Cell body (soma)** → processes the signals

- **Axon** → sends output signals
- **Synapse** → connection between neurons with adjustable strength

Artificial neurons follow the same logic:

- Inputs $\approx$ dendrites
 - Weights $\approx$ synaptic strengths
 - Summation function $\approx$ soma
 - Activation function $\approx$ firing of neuron
 - Output $\approx$ axon signal
-

3. Structure of an Artificial Neuron

An artificial neuron has 3 main components:

(i) Inputs ($x_1, x_2, x_3, \dots, x_n$)

These are numerical values representing features of data.

Example: For image data

- x_1 = pixel intensity
 - x_2 = color value
 - x_3 = texture value
-

(ii) Weights ($w_1, w_2, w_3, \dots, w_n$)

Weights determine the **importance** of each input.

- High weight → input is important
- Low weight → input is less important
- Negative weight → inhibitory effect

Weights are **learned automatically** during training.

(iii) Bias (b)

Bias is an additional parameter that shifts the decision boundary.

Role:

- Helps the neuron learn even when all inputs are zero
- Increases flexibility of the model

(iv) Summation Function

The summation function computes the **weighted sum** of inputs:

$$NET = \sum_{i=1}^n (w_i x_i) + b$$

This net input is fed into the activation function.

(v) Activation Function (ϕ)

The net input is passed through an activation function:

$$Output = \phi(NET)$$

Activation functions introduce **non-linearity**, allowing neural networks to learn complex patterns.

4. Mathematical Model of Artificial Neuron

Step 1: Compute weighted sum

$$NET = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

Step 2: Apply activation function

$$y = \phi(NET)$$

Final Output:

$$y = \phi\left(\sum w_i x_i + b\right)$$

5. Types of Activation Functions

Activation functions decide whether the neuron should fire or not.

1. Step Function

$$\phi(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Used in early perceptrons.

2. Sigmoid Function

$$\phi(x) = \frac{1}{1 + e^{-x}} \quad \phi(x) = \frac{e^x}{1 + e^x}$$

- Smooth
- Outputs in (0, 1)
- Used in binary classification

3. Hyperbolic Tangent (tanh)

$$\phi(x) = \tanh(x) \quad \phi(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output between (-1, 1)
- Zero-centered

4. ReLU (Rectified Linear Unit)

$$\phi(x) = \max(0, x) \quad \phi(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

- Most widely used in deep networks
- Solves vanishing gradient problem

5. Softmax

Used in multi-class classification:

$$\phi(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad \phi(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

6. Working Mechanism of an Artificial Neuron

1. **Receives Input:**
Inputs: $x_1, x_2, x_3, \dots, x_n$
2. **Weights and Bias Applied:**
Each input multiplied by a weight: $w_i x_i$
Bias added.
3. **Summation:**
 $NET = \sum w_i x_i + b$
4. **Activation:**
Output: $y = \phi(NET)$

5. **Transmission:**

Output passes as input to next neurons in the network.

7. **Learning in an Artificial Neuron**

Learning means adjusting weights and bias.

Learning Rules:

- **Perceptron Learning Rule**
- **Least Mean Square (LMS) Rule**
- **Delta Rule / Gradient Descent**
- **Hebbian Learning Rule**

In modern deep learning, **backpropagation + gradient descent** is the standard approach.

8. **Limitations of a Single Neuron**

A single neuron can only solve **linearly separable** problems.

Examples:

- AND → solvable
- OR → solvable
- XOR → cannot be solved by a single neuron

To solve complex problems, neurons must be arranged into **multi-layer networks**.

9. **Real-World Examples of Artificial Neurons**

Artificial neurons are used in:

- Image classification (CNN layers)
- Speech recognition
- Natural language processing
- Medical diagnosis
- Robotics and control systems

Each node in a deep learning model is an artificial neuron performing the same basic operations.

Activation Functions in Deep Learning – Detailed Explanation

An **activation function** is a mathematical function applied to a neuron's output in a neural network.

Its main purpose is to **introduce non-linearity**, allowing deep learning models to learn complex patterns and relationships from data.

Without activation functions, a neural network becomes a **purely linear model**, regardless of the number of layers.

1. Why Activation Functions Are Important

✓ Introduce Non-linearity

Real-world data (images, speech, language) is nonlinear. Activation functions help networks learn this complexity.

✓ Enable Deep Architectures

They allow stacking multiple layers and learning hierarchical features.

✓ Allow Gradient-Based Learning

Activation functions make it possible to compute gradients during **backpropagation**, which is the basis of training deep networks.

✓ Control the Output Range

Some activation functions output values between:

- 0 to 1
- -1 to 1
- or all real numbers

This helps depending on the task.

2. Types of Activation Functions

Activation functions in deep learning are divided into:

1. **Linear Activation Function**
2. **Non-linear Activation Functions**
 - Sigmoid
 - Tanh
 - ReLU family
 - Softmax
 - Swish / GELU

3. Common Activation Functions (With Explanation)

● 1. Linear Activation Function

$$f(x) = x$$

► Characteristics

- Produces output directly proportional to input
- No non-linearity
- Rarely used in hidden layers

► Used in:

- Output layer for **regression** problems
-

● 2. Sigmoid (Logistic Activation)

$$f(x) = \frac{1}{1 + e^{-x}}$$

► Output Range:

0 to 1

► Characteristics:

- Good for models where output represents probability
- Smooth and differentiable

► Disadvantages:

- Causes **vanishing gradient problem**
- Not zero-centered

► Used in:

- **Binary classification output layer**
-

● 3. Hyperbolic Tangent (tanh)

$$f(x) = \tanh(x)$$

► **Output Range:**

-1 to 1

► **Advantages:**

- Zero-centered → gradients move in a balanced way
- Stronger gradients than sigmoid

► **Disadvantages:**

- Still suffers from vanishing gradients

► **Used in:**

- Hidden layers (older networks)
-

● **4. ReLU (Rectified Linear Unit)**

$$f(x) = \max(0, x)$$

► **Characteristics:**

- Simple and computationally efficient
- Helps solve vanishing gradient problem
- Creates sparse activations (many zeros)

► **Disadvantage:**

- **Dying ReLU problem:** neurons stop learning for negative values

► **Used in:**

- Almost all modern deep learning models (CNNs, MLPs, vision networks)
-

● **5. Leaky ReLU**

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

► **Advantage:**

Allows a small slope for negative values → avoids dying ReLU.

● 6. Parametric ReLU (PReLU)

A learnable version of Leaky ReLU.

► Used in:

- Advanced deep nets
- Allows model to learn the best negative slope

● 7. GELU (Gaussian Error Linear Unit)

Used in modern Transformer models.

► Advantages:

- Smooth
- Better performance than ReLU in NLP models

► Used in:

- BERT
- GPT
- Transformers

● 8. Softmax Activation

Used in multi-class classification output layer.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad \text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

► Characteristics:

- Converts outputs into **probabilities**
- Sum of probabilities = 1

► Used in:

- Neural networks with **multiple classes**

McCulloch–Pitts Neuron (MP Neuron)

Introduced by: Warren McCulloch and Walter Pitts in **1943**

Also called: *Threshold Logic Unit (TLU) or Binary Neuron*

The **McCulloch–Pitts neuron** is considered the **first mathematical model of an artificial neuron**. It laid the foundation for modern neural networks and deep learning.

1. Introduction

The MP neuron is a **simplified mathematical abstraction** inspired by biological neurons. It models how a neuron receives input, processes it, and produces an output based on a threshold.

It assumes:

- Inputs are **binary**: 0 or 1
- Output is **binary**: 0 or 1
- Weights are **fixed** (original model did not include learning)
- Threshold decides whether the neuron “fires”

Despite its simplicity, the MP neuron demonstrated the idea that neural activity can be computed logically.

2. Structure of MP Neuron

The neuron consists of:

1. **Inputs:** $x_1, x_2, \dots, x_n$
2. **Weights:** $w_1, w_2, \dots, w_n$
3. **Summation unit**
4. **Threshold (θ)**
5. **Activation function** (step function)

Diagram (Conceptual)

$$\begin{array}{l} x_1 \text{ --- } w_1 \backslash \\ x_2 \text{ --- } w_2 \text{) ---> SUM ---> step(threshold) ---> Output (0/1) \\ \dots \quad / \\ x_n \text{ --- } w_n / \end{array}$$

3. Mathematical Model

The neuron computes a **weighted sum**:

$$S = \sum_{i=1}^n w_i x_i$$

The output y is:

$$y = \begin{cases} 1 & \text{if } S \geq \theta \\ 0 & \text{if } S < \theta \end{cases}$$

This is called a **binary step activation function**.

4. Characteristics of MP Neuron

Feature	Explanation
Binary inputs & outputs	Only 0 or 1
Threshold-based firing	Output = 1 when $\text{sum} \geq \text{threshold}$
No learning	Weights are not adjusted automatically
Deterministic	No probabilities or continuous values
Implements logic gates	Can perform AND, OR, NOT

5. MP Neuron as Logic Gates

(a) AND Gate

To output 1 only when all inputs are 1:

- Inputs: x_1, x_2
- Weights: 1, 1
- Threshold: 2

$$y = 1 \text{ if } x_1 + x_2 \geq 2$$

(b) OR Gate

Output 1 if any input is 1:

- Threshold: 1

$$y = 1 \text{ if } x_1 + x_2 \geq 1$$

(c) NOT Gate

Single input:

- Weight = -1
- Threshold = 0

$$y = 1 \text{ if } -x \geq 0$$

The MP neuron can implement **all Boolean functions expressible with AND, OR, NOT**.

6. Limitations of MP Neuron

Although revolutionary, the MP model is limited:

1. No learning mechanism

Weights have to be manually assigned.

2. Cannot solve non-linearly separable problems

Example: **XOR problem**

This limitation was critical and stalled neural network research for years.

3. Only binary output

Real-world tasks need continuous outputs (like probabilities).

4. Cannot represent time or memory

The model does not capture time-based behavior or internal states.

7. Importance and Impact

Despite limitations, the MP neuron is historically significant:

- Provided the **first theoretical basis** for artificial neural networks.
- Showed the brain can be modeled using **mathematical logic**.
- Inspired later advances:
 - Perceptron (Rosenblatt, 1957)
 - Multi-layer neural networks
 - Deep Learning architectures

It remains a fundamental concept taught in AI, ML, and neural network courses.

Perceptron – Detailed Notes

1. Introduction

The **Perceptron**, proposed by **Frank Rosenblatt in 1957**, is the **foundation of modern neural networks**.

It is the first neuron model that can **learn** from data by adjusting its weights.

It improves upon the McCulloch–Pitts Neuron by introducing:

- **Learnable weights**
 - **Bias term**
 - **Learning rule** (Perceptron Learning Algorithm)
-

2. Structure of the Perceptron

A perceptron has:

1. **Inputs:** $x_1, x_2, \dots, x_n$
2. **Weights:** $w_1, w_2, \dots, w_n$
3. **Bias:** b
4. **Summation unit**
5. **Activation function** (step function or sign function)
6. **Output:** 0/1 or $-1/+1$ depending on convention

Diagram

$x_1 \text{ --- } w_1 \backslash$
 $x_2 \text{ --- } w_2 \text{) } \rightarrow \Sigma \text{ (weighted sum) } + \text{ bias } \rightarrow \text{ activation } \rightarrow \text{ output}$
 $\dots \quad /$
 $x_n \text{ --- } w_n$

3. Mathematical Model

Weighted Sum:

$$S = \sum_{i=1}^n w_i x_i + b$$

Activation (Step Function):

$$y = \begin{cases} 1, & \text{if } S > 0 \\ 0, & \text{otherwise} \end{cases}$$

or sometimes:

$$y = \text{sign}(S)$$

4. Perceptron Learning Rule

This is what makes the perceptron *trainable*.

Weight Update Formula:

$$w_i \leftarrow w_i + \eta(t - y)x_i$$

Bias Update:

$$b \leftarrow b + \eta(t - y)$$

Where:

- t = true label
- y = predicted output
- η = learning rate ($0 < \eta \leq 1$)

How learning happens:

- If prediction is correct → **no change**
 - If prediction is wrong → weights move in direction that reduces the error
-

5. Perceptron as a Linear Classifier

A perceptron divides the input space using a **hyperplane**:

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = 0$$

This hyperplane separates data into two classes:

- Positive side → output 1
- Negative side → output 0

Thus the perceptron is a **binary linear classifier**.

6. Example: Implementing Logic Gates

AND Gate

- Inputs: (0,0), (0,1), (1,0), (1,1)
- Only (1,1) outputs 1

One solution:

- $w_1 = 1$
- $w_2 = 1$
- $b = -1.5$

OR Gate

- Outputs 1 for any input that has at least one 1

One solution:

- $w_1=1, w_2=1, b=-0.5$
- $w_1=1, w_2=1, b=-0.5$
- $w_1=1, w_2=1, b=-0.5$

NOT Gate

- $w_1=-1, w_2=1, b=0.5$

XOR cannot be solved by a single perceptron → not linearly separable.

7. Limitations of Single-Layer Perceptron

Limitation	Explanation
Cannot solve nonlinear problems	e.g., XOR
Only binary classification	No multi-class output
Uses step activation	Not differentiable; cannot use gradient descent
Learning is slow for complex data	Only works for linearly separable data

This limitation led to criticism in 1969 (Minsky & Papert), causing the first “AI winter.”

8. Multi-Layer Perceptron (MLP) – Solution

To overcome limitations, researchers introduced:

- Multiple layers (hidden layers)
- Non-linear activation functions (ReLU, sigmoid, tanh)
- Backpropagation learning algorithm

This moved neural networks into the modern deep learning era.

9. Applications of Perceptrons

- Pattern recognition
- Image classification
- Signal processing
- Simple neural network models
- Binary decision-making tasks
- Forming the basis for deep neural networks

Forward and Backpropagation in Neural Networks (Detailed Notes)

Neural networks learn by two major processes:

1. **Forward Propagation (Forward Pass)**
2. **Backpropagation (Backward Pass)**

Together, they train the network by minimizing error using gradient descent.

1. Forward Propagation (Forward Pass)

Forward propagation is the process of **passing input data through the network** to compute the **output**.

1.1 Steps in Forward Propagation

Step 1: Input Layer

- Receive input vector

$$X = [x_1, x_2, \dots, x_n] \quad X = [x_1, x_2, \dots, x_n] \quad X = [x_1, x_2, \dots, x_n]$$

Step 2: Weighted Sum at Each Neuron

For each neuron:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b \quad z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b \quad z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

This is also written as:

$$z = W \cdot X + b \quad z = W \cdot X + b \quad z = W \cdot X + b$$

Step 3: Apply Activation Function

Example using sigmoid:

$$a = \sigma(z) \quad a = \sigma(z) \quad a = \sigma(z)$$

Other activations:

- ReLU
- tanh
- Softmax
- Linear

Step 4: Output Layer

Creates the final prediction:

$\hat{y}$ = Output of the last layer
 $\hat{y}$ = Output of the last layer

1.2 Loss Calculation

Once we have predicted output $\hat{y}$, compare with true output y .

Examples:

Mean Squared Error (MSE)

$$L = \frac{1}{2} \sum (y - \hat{y})^2$$

Cross-Entropy Loss

Used in classification:

$$L = -\sum y \log(\hat{y})$$

Forward propagation ends here.

2. Backpropagation (Backward Pass)

Backpropagation is the process of **computing gradients of the loss** with respect to weights and biases, and **updating them** to reduce error.

Backpropagation uses:

- **Chain Rule of Calculus**
 - **Gradient Descent** (or variants like Adam, RMSProp)
-

2.1 Steps in Backpropagation

Step 1: Compute error at output

$$\text{Error} = \frac{\partial L}{\partial \hat{y}}$$

Step 2: Backpropagate error through activation function

$$\delta = \frac{\partial L}{\partial z}$$

For sigmoid:

$$\frac{\partial a}{\partial z} = a(1-a)$$

Step 3: Compute gradient w.r.t weights

$$\frac{\partial L}{\partial w_i} = \delta \cdot x_i$$

Step 4: Update weights

Using Gradient Descent:

$$w_i = w_i - \eta \frac{\partial L}{\partial w_i}$$

Bias:

$$b = b - \eta \frac{\partial L}{\partial b}$$

Where

- η = learning rate
- Gradients flow backward from output to input

2.2 Chain Rule in Backpropagation

Backpropagation relies on chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Each term is computed layer by layer.

3. Forward vs Backward Pass – Comparison

Aspect	Forward Propagation	Backward Propagation
Purpose	Compute output	Compute gradients
Direction	Input → Output	Output → Input
Uses	Weighted sum and activations	Chain rule and derivatives
Produces	Prediction $\hat{y}$	Weight updates
Depends on	Input & weights	Loss and forward pass values

★ Example (Simple Network)

Consider 1 hidden layer:

Forward Pass

$$\begin{aligned} z_1 &= w_1x + b_1 \\ z_2 &= w_2a_1 + b_2 \\ a_1 &= \sigma(z_1) \\ \hat{y} &= \sigma(z_2) \end{aligned}$$

Backward Pass

Compute derivative of loss:

$$\begin{aligned} \delta_2 &= (y - \hat{y}) \sigma'(z_2) \\ \delta_1 &= \delta_2 w_2 \sigma'(z_1) \end{aligned}$$

Update Weights:

$$\begin{aligned} w_2 &= w_2 + \eta \delta_2 a_1 \\ w_1 &= w_1 + \eta \delta_1 x \end{aligned}$$

🔗 4. Why Back propagation Works

- Efficiently computes gradients for all weights
- Handles multi-layer networks
- Enables deep learning
- Works with any differentiable activation

It is the algorithm that **made deep learning possible**.

Gradient Descent (GD) – Detailed Notes

Gradient Descent (GD) is an **optimization algorithm** used to minimize a **loss function** by iteratively adjusting the model parameters (weights and biases).

It is the **core training algorithm** behind:

- Neural Networks
 - Linear/Logistic Regression
 - Deep Learning models
 - Many ML algorithms
-

1. What is Gradient Descent?

Gradient Descent finds the **lowest point** of a loss function by moving in the direction of the **negative gradient** (steepest descent).

It answers the question:

→ □ *"How should weights change to reduce error?"*

2. Loss Function

Training requires a loss function $L(w)$ such as:

- **MSE** (Regression)
- **Cross-Entropy** (Classification)

Goal:

$$\min_{w} L(w)$$

Weights are updated to reduce this loss.

3. Update Rule

For each weight w :

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

Where:

- η = learning rate
- $\frac{\partial L}{\partial w}$ = gradient of loss

Negative sign is used because we move opposite to gradient direction.

4. Intuition

- Imagine standing on a mountain (loss surface).
 - Your goal is to reach the valley (minimum loss).
 - The steepest downward direction is given by the **negative gradient**.
 - Taking steps of size **learning rate (η)** moves you toward the minimum.
-

5. Types of Gradient Descent

1 Batch Gradient Descent (BGD / Full GD)

- Uses **entire dataset** to compute gradient.
- Very accurate but slow.

Update:

$$w = w - \eta \nabla L(w)$$

Pros: stable, accurate

Cons: slow, requires high memory

2 Stochastic Gradient Descent (SGD)

- Uses **one sample at a time**.
- Fast and good for large datasets but noisy.

Update:

$$w = w - \eta \nabla L(w_i)$$

Pros: very fast, works well for online learning

Cons: noisy updates → may bounce around

3 Mini-Batch Gradient Descent

- Uses a **small batch of samples** (e.g., 32, 64, 128).
- Combines stability and speed.

Update:

$$w = w - \eta \nabla L(\text{batch})$$

Pros: best in practice, used in all deep learning

Cons: requires tuning batch size

6. Learning Rate (η)

Learning rate controls step size.

If η is too small

- Slow convergence

If η is too large

- Overshooting

- Divergence

Good practice

Use adaptive optimizers:

- Adam
- RMSProp
- Momentum

▲ 7. Convergence Behavior

Gradient descent can converge to:

- **Global minimum** (convex functions)
- **Local minimum** (non-convex functions like neural networks)
- **Saddle points**

Neural networks have complex surfaces → many local minima.

8. Gradient Descent with Momentum

Momentum helps accelerate learning:

$$v_t = \beta v_{t-1} + (1-\beta) \nabla L_{v_t} \\ w = w - \eta v_t$$

Reduces oscillations and improves speed.

9. Advantages & Disadvantages

✓ Advantages

- Simple and intuitive
- Works for many ML models
- Scalable

✗ Disadvantages

- Sensitive to learning rate
- Can get stuck in local minima
- Slow for very large datasets (Batch GD)

10. Example (Simple)

Loss function:

$$L(w) = w^2$$

Gradient:

$$dL/dw = 2w \quad \frac{dL}{dw} = 2w \quad dL = 2w$$

Update:

- If $w = 4$

$$w_{\text{new}} = 4 - \eta(2 \cdot 4) = 4 - \eta(8) = 4 - 8\eta$$

As iterations continue $\rightarrow w \rightarrow 0$